

Sistemas de archivos

La memoria principal o RAM del hardware es volátil, es decir que su contenido se pierde al apagarse el sistema. Además su capacidad de almacenamiento es limitado, por lo que se necesita un medio de almacenamiento masivo y persistente de los datos y programas del sistema de los usuarios.

Los dispositivos o medios y tecnologías de almacenamiento han evolucionado en el tiempo, desde cintas o tarjetas de papel perforados cintas, cilindros o discos magnéticos y ópticos hasta las actuales memorias (mal llamados discos) no volátiles de estado sólido que toman la forma de dispositivos USB conocidos como pendrives o con interfaces compatibles con el estándar ATA. En estas notas denominaremos *disco* a un dispositivo de almacenamiento.

Estos dispositivos almacenan datos en bloques de un tamaño fijo determinado, por ejemplo 512 bytes.

Un dispositivo de almacenamiento puede dividirse lógicamente en *particiones* o *volúmenes*. Esto permite implementar sistemas de archivos que no deban manejar un inmenso espacio de números de bloques.

Podemos asumir que un dispositivo no particionado provee al menos un volumen.

Un disco se *formatea* a bajo nivel con la siguiente estructura lógica.

bs	partition table	volume 1	volume 2	...
----	-----------------	----------	----------	-----

Figura 1: Estructura lógica de un disco.

El *boot sector* (*bs*) contiene un pequeño programa de carga del kernel de un SO en memoria y transferirle el control.

La *tabla de partición* contiene información (comienzo, fin, booteable, fs-type, etc) sobre la división lógica del disco. Cada parte se denomina *partición* o *volumen*.

El estándar BIOS incluye la tabla de particiones primaria (de 4 entradas) en el *boot sector*. El nuevo estándar UEFI (como se describen [booting](#)) se basa en una estructura menos restricta.

En cada volumen se almacena un *filesystem* (*FS*).

Una plataforma de hardware puede incluir varios dispositivos de almacenamiento de diferentes tecnologías. Un SO debe proveer abstracciones para gestionar objetos de datos y programas. Este subsistema dentro de un SO se conoce como *sistema de archivos (filesystem)*.

Interface de un sistema de archivos

Este subsistema abstrae una unidad de almacenamiento en un *archivo*. Para permitir una mejor organización de la inmensa cantidad de archivos que puede tener un sistema de cómputos actual, los archivos se almacenan en *contenedores* que se denominan *directorios o carpetas*.

Un archivo se representa de la forma *(attributes,data)*.

Los atributos comúnmente son:

- *Nombre*: Para su identificación en el sistema.
- *Tipo*: Según el tipo de información que contiene, su formato lógico, etc.
- *Tamaño*: Comúnmente en bytes.
- *Dueño*: Usuario que puede definir su control de acceso
- *Creador*: Usuario que lo creó originalmente.
- *permisos de acceso*: Qué usuarios pueden acceder y cómo (lectura, escritura, ejecución, etc).
- *Hora y fecha de último acceso*.
- Otros: Fecha y hora de su creación, etc.

Los datos están almacenados en diferentes *bloques* del dispositivo pero deberán aparecer ante el usuario como una *secuencia contigua de bytes*.

Cada archivo estará contenido en un directorio. Los sistema de archivos deberán contener al menos un directorio, llamado el *directorio raíz*. En los SO tipo UNIX el directorio raíz se denota `/` , mientras que en MS-WINDOWS con `\` (*backslash*).

Así los archivos quedan representados en una estructura de árbol, donde los archivos son las hojas y los directorios los archivos intermedios.

Para identificar unívocamente a un archivo del sistema se usa su *path* o camino desde la raíz, por ejemplo `/home/os-student/books/SO-book.pdf` .

Esto permite reusar nombres de diferentes archivos en diferentes carpetas.

Algunos SOs, como DOS o MS-Windows, determinan el *tipo* del archivo por su *extensión* que comúnmente es un *sufijo* del nombre. Por ejemplo, en estos sistemas, los archivos ejecutables tienen extensión `.com` , `.exe` y otros.

En los SO tipo UNIX, el tipo está dado en los metadatos y las *extensiones* son simplemente una convención entre los usuarios pero el sistema no hace ninguna interpretación. El carácter `.` (punto) es simplemente un símbolo más permitido en los nombres (ej: `ld-linux.so.2`). Estos sistemas clasifican a los archivos en los siguientes tipos:

- *Regulares*: Archivos de datos y ejecutables.
- *Directorios*: Contenedores de archivos y sub-directorios.
- *Enlaces simbólicos*: Referencias (alias) a otros archivos.
- *Pipes y FIFOs*: Usados para IPC.
- *Especiales*: Representan dispositivos (de bloques o caracteres).
- *Sockets*: Representan un mecanismo de comunicación local o remoto.

Operaciones sobre archivos

Las operaciones sobre archivos regulares comúnmente son las siguientes:

- *Creación*
- *Lectura*: Leer una secuencia de datos.
- *Escritura*: Escribir una secuencia de datos.
- *Ejecución*: Ejecutar un programa (o script interpretado).
- *Borrado*: Eliminación (*delete*) del archivo.
- *Append*: Escribir datos al final.
- *Truncado*: Borrar datos al final.
- *Posicionado*: Mover el *cursor* de lectura/escritura en una posición determinada.
- *Renombrado*: Cambiar el nombre.
- *Locking*: Obtener acceso exclusivo.
- Cambiar otros metadatos como el *dueño*, *permisos*, etc.

Operaciones sobre directorios:

- *Creación*: En ésta operación es común que un directorio vacío se cree con dos *enlaces*: El `..` que referencia al *directorio padre* y el `.` que referencia al directorio en sí mismo (actual).
- *Búsqueda de un archivo o directorio*: Búsqueda de archivos o subdirectorios que contiene. Cada elemento en un directorio se conoce como *directory entry*.
- *Eliminación*: Comúnmente se requiere que el directorio esté vacío.

En las operaciones que usan un *path*, éste puede ser un *full path* (camino desde la raíz) o *relativo al directorio corriente*. Por ejemplo, `open("/home/user1/docs/mycv.tex")` usa un *full path*, mientras que `open("docs/mycv.tex")` refiere al mismo archivo asumiendo que el directorio corriente es `/home/user1/`.

Un programa se ejecuta en un cierto *ambiente* representado por un conjunto de *variables de ambiente*. Cada variable de ambiente es un par `variable=value` . En UNIX, un programa puede acceder a las variables de ambiente mediante el tercer argumento de `main` . El directorio corriente (donde se está parado en el shell) está representado por la variable de ambiente `PWD` (hacer `echo $PWD`).

Algunas de las llamadas al sistema sobre archivos en sistemas tipo UNIX son:

- *Creación*: `int creat(file_path, flags)` .
- *Apertura*: `int open(file_path, flags)` . Esta operación retorna un *file descriptor* para ser presentado en las subsiguientes operaciones. Posiciona el *cursor* en cero cuando se abre en modo lectura y/o escritura o al final cuando se abre en modo `APPEND` .
- *Cierre/liberación*: `int close(fd)` . Cierra el archivo dado por el descriptor `fd` .
- *Lectura*: `int read(fd, buffer, count)` . Lee desde la posición actual (*cursor*) hasta `count` bytes en `buffer` . Retorna la cantidad de bytes leídos (0 cuando el cursor estaba en *EOF*).
- *Escritura*: `int write(fd, buffer, count)` . Escribe desde la posición actual (*cursor*) hasta `count` bytes en `buffer` . Retorna la cantidad de bytes escritos.
- *Posicionado*: `int seek(fd, pos, from)` . Posiciona el cursor.
- *Eliminación*: `int unlink(path)` .
- *Creación de enlaces*: `int link(dst, name)` y `int symlink(dst, name)` . Crean un enlace duro o simbólico (resp.) `name` al archivo `dst` . Pueden ser paths relativos o full.
- *Información*: `int stat(fd, stat_buffer)` . Retorna (en `stat_buffer`) los metadatos del archivo.

Algunas de las llamadas al sistema para directorios en UNIX son:

- *Creación*: `int mkdir(path, mode)` .
- *Eliminación*: `int rmdir(path)` .

Para acceder al contenido de un directorio, éste puede verse como un archivo que contiene registros que contienen los metadatos de cada archivo que contiene. Cada SO comúnmente ofrece funciones en la biblioteca estándar para manipular directorios. En sistemas tipo UNIX, ver la función `readdir()` .

VFS

Los SO modernos soportan diferentes tipos (o formatos) de sistemas de archivos en diferentes dispositivos de diferentes tipos y tecnologías.

En estos casos se debe presentar al usuario un sistema de archivos *uniforme* con operaciones que operen de manera transparente ocultando los detalles de bajo nivel dependientes de cada tipo de sistema de archivos y/o dispositivos.

Este modelo *uniforme* se conoce como *common file model*. En los sistemas UNIX este modelo se basa en nodos índices (descritos más abajo).

Esto se logra mediante un *virtual file system (VFS)*. El VFS, ante una llamada al sistema sobre archivos determinará sobre qué dispositivo y sistemas de archivo concreto deberá realizar esta operación. El diseño del VFS se muestra en el siguiente diagrama.

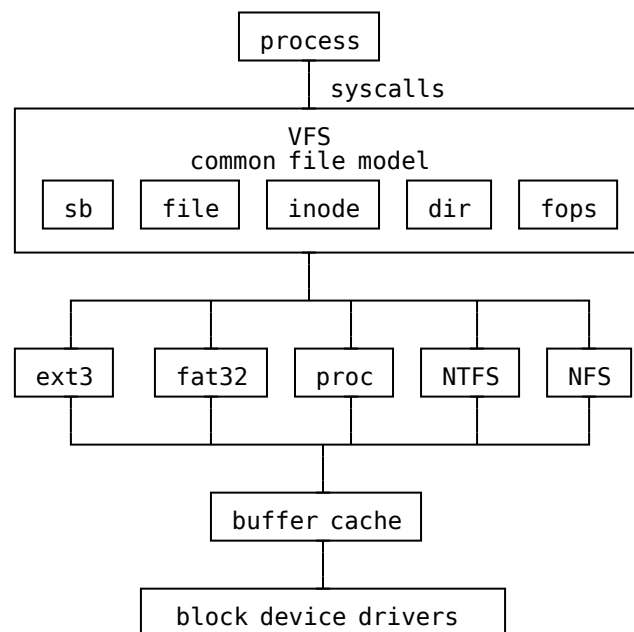


Figura 2: *Diseño del virtual file system.*

El VFS define una interface que cada FS concreto debe implementar. Esta API se implementa de manera similar que en programación orientada a objetos. El VFS define las estructuras de datos comunes que ocultan los detalles de implementación de los sistemas de archivos reales o concretos.

La interface se conoce como el *common file model* que en general representa un sistema de archivos abstracto. Cada implementación concreta de un filesystem deberá *mapear* la representación de archivos, directorios y sus operaciones al modelo abstracto.

Cada sistema de archivos real deberá representar en el VFS los metadatos de su formato específico, comúnmente almacenado en el *superblock (sb)*. El superblock contiene el tipo o id del sistema de archivos, el número de bloques, su tamaño, el lugar del directorio raíz, etc.

La estructura (objeto) *file* representa un archivo abierto por algún proceso. Comúnmente se representa como una estructura con al menos los campos *inode* (un puntero al inode que representa al archivo) y el *offset* que es un *cursor* para determinar la posición en el archivo a aplicar la siguiente operación de lectura o escritura.

Un inode contiene los *metadatos* de un archivo, como su tamaño, dispositivo donde reside, bloques de datos usados, etc.

La estructura *directory (dir)* representa un directorio, comúnmente es un par (*name*, *inode_number*). Un directorio es un archivo de datos que contiene una secuencia de esas entradas (*dir entry*). También el VFS define una interface abstracta de operaciones sobre directorios y búsqueda por *file paths*.

La estructura *fops* representa una interface abstracta que cada filesystem concreto debe implementar con las operaciones sobre archivos como `inode=open(path,mode)` , `read/write(inode,buffer,count)` , `free(inode)` , etc.

Además provee varios servicios comunes como los siguientes.

- Punto de entrada de las llamadas al sistema como `open` , `read` , `write` , ...
- *Buffers de cache*: Usado comúnmente para dispositivos de bloques. Los bloques de datos se mantienen en una *caché* para reducir el número de operaciones de entrada/salida con los dispositivos. Esa caché se basa en un *pool* limitado, por lo que comúnmente se utilizan algoritmos de reemplazo (como LRU) similares a los vistos en [memoria virtual](#). Normalmente no se usa caché sobre dispositivos de caracteres.
- *Gestión de los requerimientos de entrada/salida*: Se basan en colas de requerimientos y se usan algoritmos de ordenamiento como el del elevador para los discos. En dispositivos como *ttys* se aplica comúnmente una disciplina FIFO.
- *Manejo de la registración de los sistemas de archivos concretos*.
- *Registración de los device drivers*.
- *Montaje/desmontaje de sistemas de archivos concretos*.

Se debe notar que el módulo de *buffers en caché* es el que se comunica con los *block device drivers*. Los sistemas de archivos usan sus servicios para leer/escribir en bloques de datos.

Representación de archivos y directorios

Para representar y gestionar archivos y directorios se necesitan estructuras de datos de control y metadatos almacenados en el mismo dispositivo. Mínimamente se requiere:

- Metadatos del FS: Ubicación del bloque del directorio raíz, número de bloques en el dispositivo, etc.
- Conjunto de bloques libres y usados.
- Descripción de espacios de bloques de datos y otros para metadatos.

La forma que toman esas estructuras de datos se llama el *formato del sistema de archivos*. Crear un sistema de archivos inicial se conoce como *formatear* el dispositivo o volumen. Ver los comandos `mkfs` en sistemas tipo UNIX o `format` en sistemas DOS o MS-Windows.

Aquí analizaremos dos formatos ampliamente usados.

FAT

EL formato FAT o *file allocation table* fue desarrollado para el sistema DOS y se usó en las primeras versiones de MS-Windows. Se basa en el siguiente diagrama.

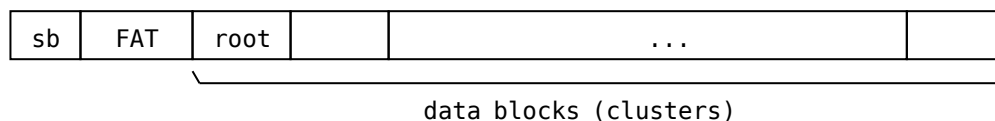


Figura 3: *Formato FAT.*

El *superblock (sb)* contiene metadatos como el tipo de FS (*FAT magic number*), el número de bloques del volumen, el tamaño de bloque, entre otros.

Para reducir el número de bloques y la fragmentación de datos, los bloques físicos se agrupan en *clusters*: grupos de bloques contiguos. Un tamaño típico de un cluster es de 1, 2, 4, 8 o 16 bloques. Un cluster es una unidad mínima de asignación de datos para un archivo.

El primer cluster de datos contiene el *directorio raíz*.

De aquí en adelante usaremos de forma intercambiable los términos *bloque* o *cluster*, significando la unidad mínima de asignación de bloques de datos a un archivo.

La *FAT* es una tabla (arreglo `fat[N]`) que tiene tantas entradas como el número de clusters de datos a usar. La FAT implementa múltiples listas enlazadas de los bloques de datos de cada archivo.

Un directorio se representa como un archivo que contiene una secuencia de registros de la forma

file name	attrs	cluster1
-----------	-------	----------

Cada entrada es de 32 bits (en FAT32) y contiene 12 bytes para el nombre del archivo, bits de atributos (size, read only, hidden, hora y fecha de creación y modificación, etc).

El campo *cluster1* contiene el número del primer cluster de datos (primer elemento de la lista). Los demás clusters se *indexan en la FAT*. La entrada `FAT[cluster1]` contiene el *next cluster*. Así un archivo que contiene tres clusters de datos se representa como en el siguiente diagrama.

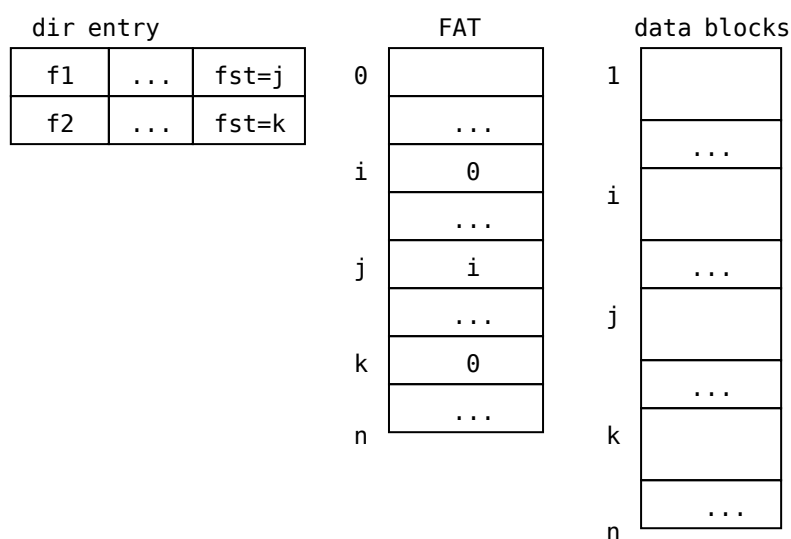


Figura 4: Estructura de un FS basado en FAT.

La figura de arriba muestra la representación de dos archivos. El archivo *f1* tiene dos clusters de datos. El primero, *j*, está referenciado en la entrada del directorio. El siguiente cluster está referenciado en $FAT[j]=i$. Este es el último ya que en $FAT[i]=0$.

El archivo *f2* usa sólo el bloque *k*.

La FAT incluye las listas de clusters de datos de cada archivo. Los clusters libres pueden inferirse de la FAT. Es suficientemente compacta para mantenerla en memoria (la cual periódicamente se salva en disco).

La gestión de bloques libres puede hacerse fácilmente. Los bloques no alcanzables de la FAT son libres. No hace falta almacenar esta información en disco ya que puede calcularse en el inicio y luego mantener una lista de bloques libres en memoria.

Nodos índices (*inodes*)

Otra técnica de representar archivos se basa en que los metadatos estén separados de la entrada del directorio. Un inode incluye la lista de bloques de datos usados por el archivo. Esta información se almacena en una estructura de datos conocida como *index node* o *inode*.

La mayoría de los sistemas de archivos para grandes volúmenes de datos se basan en esta estructura, como por ejemplo, *ext2/3/4* en GNU-Linux y otros sistemas tipo UNIX.

El sistema tiene un conjunto de *inodes* reservados en el filesystem como se muestra en la siguiente figura.

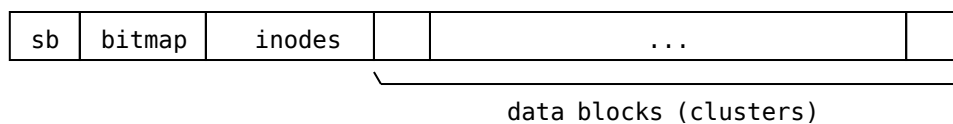


Figura 5: *Filesystem con i-nodes.*

Un *inode* tiene los atributos como el usuario dueño del archivo, permisos de acceso, tiempos de último acceso, tamaño del archivo en bytes y otros. Además contiene un arreglo o tabla de números de bloques de datos que representa la secuencia de bloques que ocupa el archivo. Esta tabla tiene un tamaño predefinido.

Un directorio es un archivo estructurado en donde cada entrada consta sólo de dos campos: (*filename, inode*). Comúnmente el directorio raíz está representado por el primer *inode*. Los *inodes* están numerados o indexados.

La siguiente figura muestra la representación de un archivo con *inodes*.

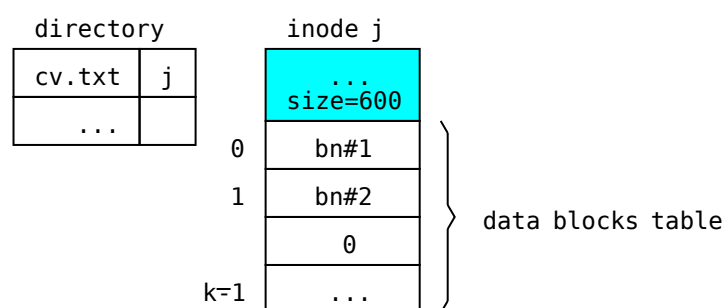


Figura 6: *Representación de archivos y directorios con inodes.*

La figura 6 muestra la representación de un directorio con la entrada para el archivo *cv.txt*, la cual también contiene el número de *inode* que tiene todos los metadatos del archivo.

El *inode* tiene los atributos y la tabla (de tamaño *k*) con los índices de los bloques de datos del archivo. En este ejemplo, los datos del archivo están en el bloque *bn#1* seguido por los datos del bloque *bn#2*.

En ésta representación, al momento de la creación (*mkfs*) del filesystem, se deberá decidir la cantidad de *inodes* a reservar en el espacio del disco. El número de *inodes* determina la cantidad máxima de archivos y directorios que puede contener el sistema.

Es conveniente usar un *bitmap* para la gestión de *inodes* y bloques de datos libres. Si bien es posible determinar los *inodes* y bloques de datos libres por medio del complemento de los alcanzables, es mas costoso en términos de acceso a disco ya que la información está distribuida en el conjunto de *inodes*.

Un *inode* representa un archivo con a lo sumo *k* bloques de datos. Comúnmente se elige, por eficiencia, el tamaño de un *inode* al tamaño de un bloque físico (sector) del disco.

Para soportar archivos grandes, con más de k bloques, comúnmente se usa la técnica del uso de *inodes indirectos*. La idea es que las últimas entradas de la tabla de números de bloques de datos apunten a otros *inodes* que sólo contienen una tabla de números de bloques de datos, como se muestra en la siguiente figura.

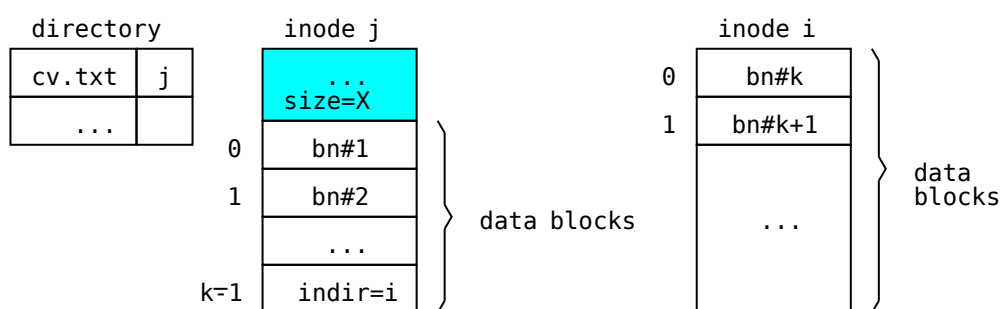


Figura 6: Representación de archivos y directorios con inodes indirectos.

En la figura de arriba la última entrada de la tabla de índices se usa como referencia a un *inode indirecto simple*, el cual sólo contiene índices a bloques de datos adicionales.

Comúnmente las entradas al final de la tabla del *inode* apuntan a inodes *simple/dobles o triples indirectos* (árboles de uno, dos o tres niveles).

Montaje de sistemas de archivos

Algunos SO, como MS-Windows, presentan en forma independiente a cada sistema de archivos de cada dispositivo. Un *full path* toma la forma `device:<path>`. Cada dispositivo se nombra usando una letra. Comúnmente el disco primario toma el nombre de dispositivo `C`.

Otros sistemas, como los del tipo UNIX, presentan al usuario un único árbol de archivos. La raíz se corresponde al sistema de archivos del dispositivo de almacenamiento primario (configurado durante el inicio del sistema). Si se quiere acceder a un sistema de archivos en otro dispositivo hay que *montarlo* en un directorio (por ejemplo en `/mnt/mypendrive`) del árbol. Ver los comandos `mount` y `umount`.

El mecanismo de montaje permite que las aplicaciones accedan a archivos con nombres pre-establecidos sin necesidad de nombrar dispositivos (como `C:`, o `D:`) facilitando la escritura de aplicaciones portables ya que los nombres de dispositivos pueden cambiar en diferentes computadoras.

A modo de ejemplo, al introducir un *pendrive* con un sistema de archivo con formato FAT (descrito más abajo), un sistema como GNU/Linux puede montarlo automáticamente (comúnmente en `/media/XXX`, donde `XXX` es un directorio generado por el sistema), dependiendo de su configuración. En el caso de no hacerlo, el usuario puede hacer (sabiendo el dispositivo aparece como `/dev/sdb`):

```
mkdir /mnt/mypendrive
mount /dev/sdb /mnt/mypendrive
```

Esta operación *cuelga* el FS del pendrive en el directorio `/mnt/mypendrive` del FS raíz como se muestra en la siguiente figura.

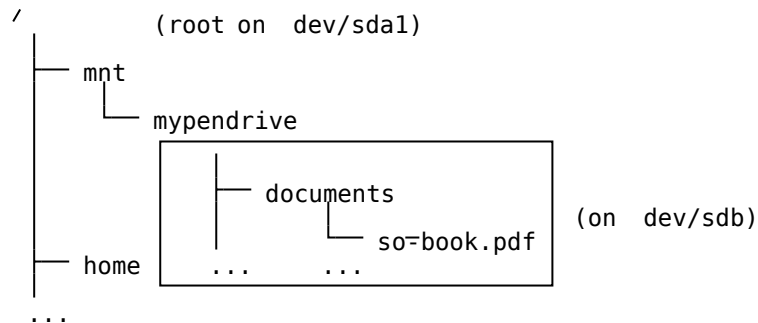


Figura 3: Sistema de archivos montado.

El usuario puede acceder a los archivos en el pendrive mediante el path `/mnt/mypendrive/`.

El VFS mantiene una tabla de sistemas de archivos montados como en el siguiente ejemplo.

mount point	fs type	device	major	access mode
/	ext4	/dev/sda1	1	rw
/mnt/mypendrive	fat32	/dev/sdb	5	r
/proc	procfs	proc	0	rw
...	...	...	...	

En sistemas tipo UNIX, esta información puede obtenerse usando el comando `mount` o viendo el contenido del archivo `/etc/mtab`.

El comando `umount /mnt/mypendrive` desmontará el archivo. Luego de esto es seguro retirarlo ya que el desmontado hace un *flush* de los bloques que podrían aún estar en caché al dispositivo.

El VFS define una interfaz que cada FS concreto debe implementar y registrar con las siguientes operaciones y estructuras de datos:

- *mount*: Invocada cuando se monta el FS. Debe cargar y registrar el *super block*.
- *umount*: Invocada al desmontar el FS. Libera el super block.
- *Operaciones sobre directorios*: Creación, destrucción, búsqueda por *file paths*, etc.
- *Operaciones sobre archivos (inodes)*.

Cada proceso registra sus archivos abiertos en una tabla en su propio descriptor. Comúnmente se representa como un arreglo o lista de punteros a entradas en la tabla de `struct file`, como el campo `ofile[]` en `struct proc` de xv6.

Aquí se ha descrito un VFS que define el *common file model* basado en *inodes*. Un sistema de archivos de tipo FAT deberá *mapear* la FAT a los *inodes*.

La tabla de montaje permite al VFS determinar con qué sistema de archivos concreto interactuar.

Por ejemplo, ejecute la llamada al sistema `fd=open("/home/user1/docs/mycv.tex", ...)` el VFS realizará las siguientes operaciones:

1. Determinar en qué FS concreto se encuentra el archivo: En éste caso encuentra en el sistema de archivos raíz, que es de tipo `ext4`.
2. Se invoca una función del tipo `inode=fs.dir_lookup("/home/user1/docs/mycv.tex")` la cual busca y lee el inode correspondiente al archivo.
3. EL VFS crea una nueva entrada de una estructura `file` en la tabla de archivos abiertos del sistema.
4. Registra en el descriptor del proceso el nuevo archivo abierto.
5. Retorna el índice en `ofiles[]` al proceso el cual representa el *file descriptor*.

Luego, cuando el proceso ejecute una llamada al sistema del tipo `write(fd,buffer,count)`, el VFS

1. Accede a la estructura `file` correspondiente a `fd`.
2. De la tabla de montaje se determina el FS concreto a usar. En este caso es el módulo `ext4`.
3. Invoca a una función de la forma `fs.write(inode, buffer, count, offset)`. Esta función requerirá escribir en uno o más bloques. Este servicio lo provee la capa de *buffer de caché* en una función del tipo `bio_write(buf,dev)`.
4. Si `buf` no está en la caché, hay que cargarlo del disco en un nuevo buffer de caché. Luego, se modifica el buffer de caché y se marca como *dirty* para que posteriormente se escriba en disco (ej: en un `close(fd)` o cuando haya expirado un timer).
5. Si el buffer de cache requiere hacer un requerimiento a disco (lectura/escritura) de un bloque, éste se encola (según una política de scheduling como la del elevador).
6. El *I/O scheduler* eventualmente tomará este requerimiento e invocará a `block_dev.do_request(request)`. Esta operación es una función del *device driver* `block_dev` registrado previamente. Es posible determinar cuál el el device driver por medio del *major number* que se encuentra en el campo *inode* de *request*. También en el *inode* está el *minor number*, así el *device driver* sabe a qué dispositivo concreto debe enviar los comandos correspondientes al requerimiento. Recordemos que un device driver puede controlar a varios dispositivos del mismo tipo (*major number*).

Algoritmo del elevador

Este algoritmo se usa en los ascensores y puede aplicarse a discos magnéticos. Para minimizar los movimientos del cabezal de lectura/escritura de un disco este algoritmo mantiene el par *(cylinder,direction)* donde *cylinder* es la posición actual del cabezal y *direction* (*left/right*) indica el sentido de su movimiento. Los requerimientos se ordenan por cercanía en base al cilindro y la dirección indicada. Primero se procesan los requerimientos más cercanos en el sentido actual de movimiento hasta llegar al final del recorrido (cilindro extremo o no hay más requerimientos en el sentido actual). Luego se invierte el sentido del movimiento y así sucesivamente.

En el caso que un proceso ejecute `fd=open("/mnt/mypendrive/documents/so-book.pdf",...)`, el VFS debe

1. Determina que en `/mnt/pendrive` hay un FS de tipo FAT montado usando la tabla de montajes.
2. Invocar a `inode=fs.dir_lookup("/documents/so-book.pdf")`, siendo `fs` la estructura *filesystem* correspondiente al FAT FS. Allí `documents` está contenido en la raíz del dispositivo `/dev/sdb`.
3. Crear una entrada en la tabla de archivos abiertos, registrarlo en el descriptor del proceso (`ofiles[]`) y retornar el índice correspondiente como `fd`.

Abstracción: Dispositivos como archivos especiales

En muchos SO los dispositivos *aparecen* en el sistema de archivos como **archivos especiales**. En los SO tipo UNIX aparecen en el directorio `/dev` (por *devices*).

Estos archivos permiten al VFS mantener una relación entre los dispositivos y los *device drivers* correspondientes.

Cada archivo especial contiene un par de campos numéricos: *major* y *minor numbers*. El primero determina el tipo de dispositivo. El segundo identifica un dispositivo de ese tipo. El mayor number se usa para relacionar una clase de dispositivos con su device driver.

Cada device driver se *registra en el VFS* en su inicio, indicando el archivo especial y el mayor number. Así el VFS mantiene una tabla de la forma:

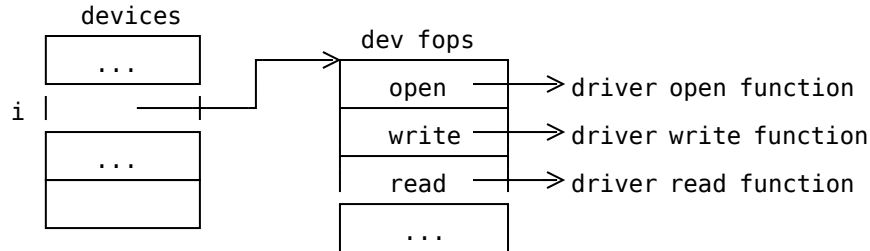


Figura 4: *Tabla de device drivers registrados.*

Cada entrada de la tabla de dispositivos se indexa por el *major number* asignado al dispositivo. La tabla asocia el dispositivo con las operaciones *registradas* por el device driver.

Así el VFS ante una operación sobre un archivo, cuyo *inode* contiene el *major number*, sabe a qué device driver realizar el requerimiento de I/O.

Descriptor de archivo

La llamada al sistema `open` retorna un descriptor de un archivo, el cual queda registrado en la *opened files table (ofiles)* del descriptor del proceso. Esta tabla se puede representar como un arreglo de punteros a estructuras de la siguiente forma:

```

struct file {
    union {
        struct pipe* pipe;
        struct inode* inode;
    };
    int ref_count;           // processes working with this file
    unsigned int offset;    // current read/write position
};

```

Un *inode* se puede representar con una estructura de la forma

```

struct inode {
    unsigned int num; // inode number (on disk)
    int ref_count;
    // next fields are retrieved/saved from/to disk
    short int type;    // file, directory, FIFO, device, ...
    short int major;
    short int minor;
    short int links;    // number of symlinks to this
    // other metadata (owner id, group id, access permissions, ...)
};

```

```
unsigned int size; // in bytes
unsigned int data_blocks[N];
};
```

Operaciones con dispositivos de caracteres

La entrada/salida con dispositivos de caracteres se diferencia con los dispositivos de bloques en dos aspectos:

1. No tienen un filesystem asociado.
2. La entrada/salida no pasa por la caché de bloques (*no buffered*), sino que se hace *directamente* al/desde el dispositivo.

A modo de ejemplo, cuando un proceso desea escribir en el descriptor 1, (salida estándar) éste descriptor refiere a una `struct file` en la cual el campo `inode` describe un *archivo especial* (ej: `/dev/tty` en Linux o `console` en xv6) representando un dispositivo de caracteres. El *major number* identifica el driver con el que se debe interactuar.

El VFS deberá invocar a la función correspondiente del *device driver* mediante `devices[p->ofiles[1]->inode.major].write(p->ofiles[1], ...)`.

En xv6 (similarmente en los sistemas UNIX), los *character device drivers* se *registran* en la tabla `struct devsw devsw[]`. En la versión xv6-riscv sólo hay un dispositivo registrado: la *consola*. La comunicación con éste dispositivo se hace por medio del *puerto serie UART*. El archivo `uart.c` implementa el *bottom half* (bajo nivel) y en `console.c` la lógica de alto nivel (*tty discipline*) del *device driver* de consola.

Se debe notar que el archivo `console` en la raíz del sistema de archivos es un archivo especial, que representa al dispositivo. Este archivo es abierto por `init` tres veces, para la entrada, salida y salida de errores estándar. Estos descriptors luego son *heredados* por los demás procesos (shell y otros) por la llamada al sistema `fork()`.

Filesystem de xv6

Xv6 sólo monta un único sistema de archivos que tiene su propio formato basado en inodes. El sistema de archivos se construye al compilarse los procesos de usuario mediante la utilidad `mkfs` incluida en el `build-system`.

La implementación de filesystem es por capas.

File descriptor	file.h/c
Pathname	} fs.h/c
Directory	
Inode	
Logging	log.c
Buffer cache	bio.c
Disk driver	virtio=disc.c

Figura 5: *Capas del filesystem de xv6.*

El sistema de archivos es *transaccional* para evitar inconsistencias en *crashes* o cortes de energía. Antes de escribir un conjunto de requerimientos a disco, xv6 los escribe en un *log* (área especial). Luego realiza un *commit*, replicando (o moviendo) las operaciones del log a los bloques de disco correspondientes. Finalmente borra el log.

Cuando el sistema inicia, el FS verifica el log. Si está marcado como completado, se realizan las copias a los bloques de datos de los contenidos del log. Sino, el log se ignora y se elimina. En éste caso el sistema perderá las últimas escrituras pero queda en un estado consistente.

El buffer de cache consiste en una lista doblemente encadenada de bloques (buffers). Los buffers de cache son fijos (arreglo declarado estáticamente) y no implementa una técnica de reemplazo.

El manual de xv6 describe en mayor detalle su diseño e implementación.

Booting

El proceso de inicio de un sistema (*boot*) comúnmente comienza por un programa en ROM conocido como *firmware*.

Este firmware en la PC se denomina *Basic Input-Output System (BIOS)*. La BIOS consiste en un conjunto de programas que permite verificar la memoria RAM física instalada, algunos dispositivos básicos, como teclado y discos. En algunas arquitecturas se almacena en ROM o EPROM. Usa una memoria volátil (mantenida encendida con una pequeña batería) conocida como CMOS en la cual se mantiene la fecha y hora del sistema y algunos parámetros de configuración del hardware.

La BIOS carga en la memoria el primer sector o bloque del disco primario del sistema y le transfiere el control. Este programa *boot loader* se encarga de cargar en memoria el código y datos del kernel del SO para luego transferirle el control.

Este programa es muy pequeño ya que debe almacenarse en un bloque comúnmente de 512 bytes. Este bloque (o sector) también puede contener la *tabla de particiones* del dispositivo. El programa debe saber dónde se encuentra el kernel en el disco o volumen.

Es común que cada instalador de un SO salve en el *boot sector* un *boot loader* a medida.

Un *boot loader genérico multiboot* como por ejemplo [GRUB](#) permiten iniciar diferentes sistemas operativos instalados en diferentes dispositivos o volúmenes de un sistema.

Realizan ésta tarea en dos pasos. Se conocen como *boot loaders* de dos etapas.

En la primera etapa el *boot loader* del *boot sector* carga el *multiboot loader* (ej: GRUB) y le transfiere el control a su punto de inicio. Luego éste último, el cual es una aplicación que conoce formatos de filesystems y de ejecutables (ELF, COFF, etc), lee un archivo de configuración y presenta un menú al usuario para que seleccione el OS a iniciar.

Estos *multiboot loaders* se usan para iniciar sistemas operativos tipo UNIX (como GNU/Linux) ya que el kernel es un archivo más y no se conoce a priori su ubicación en el sistema de archivos.

El método descrito se conoce como *BIOS booting*. Esto se desarrolló en 1975 principalmente para la PC y el [Disk Operating System \(DOS\)](#).

Actualmente es común el soporte del estándar [Unified Extensible Firmware Interface \(UEFI\)](#) el cual se basa en un mecanismo mucho más flexible. Un disco para que sea *bootable* deberá contener una partición del tipo FAT rotulada *EFI*. En esa partición se pueden almacenar directamente los kernels de los SO o arrancadores (como GRUB) a iniciar y los archivos de configuración correspondientes. El firmware UEFI conoce el formato FAT y puede cargar diferentes OSs en base a sus archivos de configuración.

El estándar define también en qué áreas de memoria el kernel encontrará información sobre la configuración del hardware.

Cada kernel (o boot loader) es un archivo con extensión `efi` debe ser compilado y enlazado apropiadamente para soportar el estándar.

También el estándar UEFI soporta que los kernels se firmen para verificar su integridad. El firmware permite almacenar en la EPROM certificados con las claves públicas de la empresa u organización que los desarrolla. Esto se conoce como *secure boot*.

Entre otras características, UEFI soporta booteo remoto y tablas de partición de tamaño arbitrario basado en el estándar *Guided Partition Table (GPT)*.

Sistemas de archivos especiales

Algunos SO modernos como GNU-Linux soportan sistemas de archivos que no necesariamente su contenido se almacena en disco.

Se conocen como *pseudo filesystems* o *especiales*.

Por ejemplo, en GNU-Linux se usa el *procfs* comúnmente montado en `/proc`. Este sistema de archivos brinda información sobre datos del kernel y de cada proceso en ejecución en forma de un árbol de directorios y archivos.

Los usuarios pueden acceder y modificar datos (variables) internos del kernel como si fueran archivos.

Por ejemplo, el siguiente comando `cat /proc/cpuinfo` muestra los detalles de las cpus del sistema. El comando `cat /proc/meminfo` muestra la siguiente salida:

```
MemTotal:      994548 kB
MemFree:       65228 kB
MemAvailable:  263724 kB
Buffers:       21396 kB
Cached:        304440 kB
SwapCached:    25260 kB
Active:        267424 kB
Inactive:      503720 kB
Active(anon):  110956 kB
Inactive(anon): 351176 kB
...
```

Es posible modificar algún parámetro del kernel mediante el comando `echo <value> > /proc/sys/<class>/<parameter>`.

Otro sistema de archivos virtual comúnmente usado es el *sysfs* el cual está diseñado específicamente para contener archivos especiales que representan dispositivos, comúnmente montado en `/dev`.

Estos sistemas de archivos simplifican la manipulación de parámetros del kernel y dispositivos (y drivers). En sistemas tipo BSD-UNIX o Linux, esto también se logra mediante el comando (y syscall) `sysctl`.

Otros sistemas de archivos se implementan en memoria principal, como el *ramfs*. Estos sistemas de archivos comúnmente se usan durante el *booting* como sistema de archivos raíz inicial. Esto permite que el *boot loader* cargue en memoria la imagen del sistema de archivos en memoria y se lo presente al kernel como un *initramfs*. Esos filesystems comúnmente contienen un conjunto reducido de programas de inicio y montaje de otros sistemas de archivos en disco.

En Linux, el *initramfs* comúnmente contiene un sistema de archivos con las utilidades del paquete [busybox](#), el cual contiene versiones mínima de las utilidades más comunes de UNIX en un único ejecutable.

Busybox también se usa generalmente en *embedded systems* ya que permite generar un sistema de archivos muy pequeño como algunos archivos de configuración en [/etc](#) , dispositivos en [/dev/](#) y un kernel (generalmente linux).

[< Anterior](#)

Entrada/salida

[Próximo >](#)

Usuarios y seguridad